

Práctica 1: Señales de tráfico

ALBERTO FIERRO LEVA

JUAN GÓMEZ REY

SERGIO MUÑOZ LAUREIRO

Introducción	2
Resultados:	2
IoU > 0.5	2
IoU > 0.7	2
Estructura del proyecto.	3
Uso	3
Opciones	3
Pipeline de detección (MSER)	3
Parámetros MSER	8
Pipeline de detección (c_d - Color y Distribución Espacial)	9
Parámetros c_d	11
Archivos principales	11

Introducción

Esta es la primera práctica de la asignatura “Visión artificial” de la carrera “Ingeniería informática” en la URJC. El objetivo a desarrollar es la detección de carteles de tráfico azules típicos de autovías, para ello se ha creado un programa en python que cuenta con los siguientes dos detectores: Mser y de color+distribucion espacial.

Resultados:

IoU > 0.5

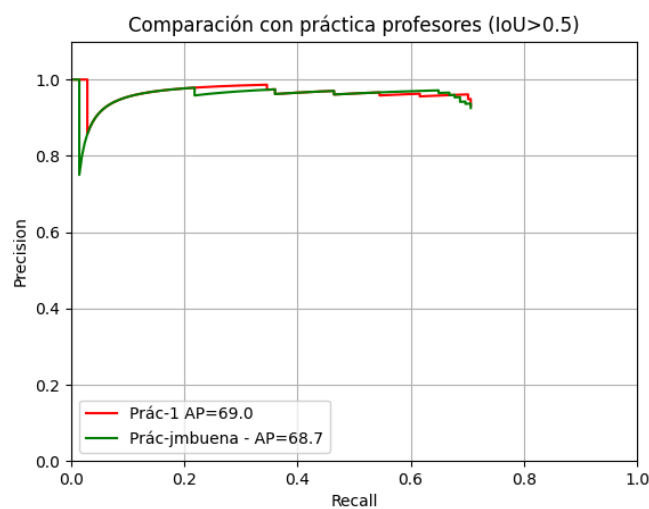


Figura 1: Comparación con resultados de profesores

IoU > 0.7

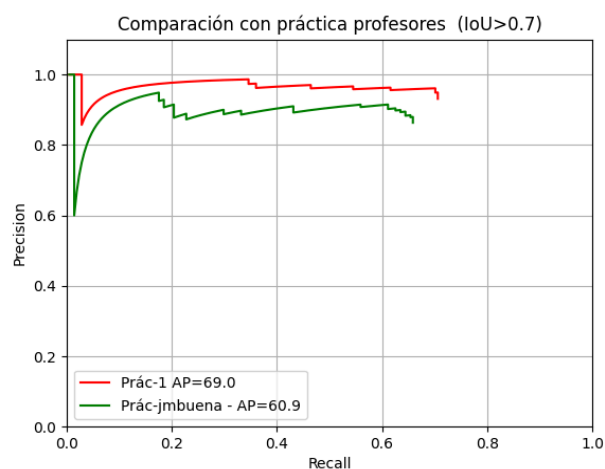


Figura 2: Comparación con resultados de profesores

Estructura del proyecto.

La estructura que sigue nuestros directorios es la siguiente:

- main.py # Punto de entrada, selecciona el detector
- evaluar_resultados.py # Evaluación de resultados (IoU)
- gt.py
- detectors/
 - detector_mser.py
 - detector_color_distribucion.py # Detector por color y distribución espacial
- img/ # Imágenes de resultados y gráficas
- resultado_imgs/
- train_detection/ # Datos de entrenamiento
- test_detection/ # Datos de test

Uso

Ejecutar detector MSER (por defecto)

```
python main.py --detector mser
```

Ejecutar detector por color y distribución espacial

```
python main.py --detector c_d
```

Opciones

Argumento	Descripción	Valor por defecto
--detector	Nombre del detector a usar	mser
--train_path	Directorio de entrenamiento	train_detection
--test_path	Directorio de test	test_detection

Pipeline de detección (MSER)

El detector en detectors/detector_mser.py implementa un pipeline de 7 pasos:

1. Preprocesamiento:

- Conversión a escala de grises
- Desenfoque Gaussiano (kernel 3x3)
- Ecualización CLAHE (clipLimit=0.5, tileGridSize=10x10)

```
def preprocesar_imagen(image):
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    gray_blur = cv2.GaussianBlur(gray_image, (3, 3), 0)
    clahe = cv2.createCLAHE(clipLimit=0.5, tileGridSize=(10,10))
    equalized_image = clahe.apply(gray_blur)
    return gray_image, equalized_image
```

2. Detección de regiones MSER:

- delta=1, min_area=5000, max_area=200000, max_variation=0.01

```
def obtener_candidatos(detector, equalized_image):
    _, bboxes = detector.detectRegions(equalized_image)

    candidatos = []
    for x, y, w, h in bboxes:
        ratio = w / float(h)
        if bottom_ratio < ratio < top_ratio:
            candidatos.append((x, y, w, h))

    return bboxes, candidatos
```

3. Filtrado geométrico:

- Eliminación por relación de aspecto ($0.2 < \text{ratio} < 9.0$)

4. Agrandado de bounding boxes:

- Expansión del 5% manteniendo el centro

```
def agrandar_caja(caja, porcentaje):  
    x, y, w, h = caja  
  
    factor = 1.0 + porcentaje  
    nuevo_w = int(round(w * factor))  
    nuevo_h = int(round(h * factor))  
  
    centro_x = x + w / 2.0  
    centro_y = y + h / 2.0  
  
    nuevo_x = int(round(centro_x - nuevo_w / 2.0))  
    nuevo_y = int(round(centro_y - nuevo_h / 2.0))  
  
    return nuevo_x, nuevo_y, nuevo_w, nuevo_h
```

5. Filtrado por color (HSV):

- Rango azul: [90, 50, 40] - [150, 255, 255]

```

def calcular_score_azul(recorte, config_color):
    if recorte.size == 0:
        return None

    recorte_resized = cv2.resize(
        recorte,
        (config_color["ancho"], config_color["alto"]),
        interpolation=interpolation
    )

    hsv_image = cv2.cvtColor(recorte_resized, cv2.COLOR_BGR2HSV)

    #Mascara binaria de los azules si el pixel esta fuera del rango es 0
    mascara_azul = cv2.inRange(
        hsv_image,
        config_color["azul_bajos"],
        config_color["azul_altos"],
    )

    # Pasamos a 0.0 - 1.0 para calcular mas facilmente el calculo posterior
    mascara_normalizada = mascara_azul.astype(np.float32) / 255.0
    correlacion = np.sum(mascara_normalizada * config_color["mascara_ideal"])
    return correlacion / float(config_color["ancho"] * config_color["alto"])

```

- Score de correlación con máscara ideal (100x50)
- Umbral de score: 0.6

```

def filtrar_detecciones_por_color(image, candidatos, config_color):
    detecciones_finales = []

    alto_imagen, ancho_imagen = image.shape[:2]

    for x, y, w, h in candidatos:
        x, y, w, h = agrandar_caja((x, y, w, h), resize_percentage)
        x1 = max(0, x)
        y1 = max(0, y)
        x2 = min(ancho_imagen, x + w)
        y2 = min(alto_imagen, y + h)

        recorte = image[y1:y2, x1:x2]
        score = calcular_score_azul(recorte, config_color)

        if score is None:
            continue

        if score > config_color["umbral_score"]:
            detecciones_finales.append((x1, y1, x2, y2, score))

    return detecciones_finales

```

6. Non-Maximum Suppression (NMS):

- Eliminación de detecciones solapadas (IoU > 0.5)
- Ordenación por score descendente

7. Salida:

- resultado.txt: <nombre>;<x1>;<y1>;<x2>;<y2>;<tipo>;<score>
- resultado_imgs/: Imágenes con rectángulos verdes y score

```
def nms_maximos_locales(detecciones, umbral_iou=umbral_iou):
    if not detecciones:
        return []

    # 1. Ordenar por SCORE de mayor a menor (priorizamos confianza sobre tamaño)

    detecciones = sorted(detecciones, key=lambda x: x[4], reverse=True)

    seleccionadas = []
    while len(detecciones) > 0:
        actual = detecciones.pop(0)
        seleccionadas.append(actual)

        # 2. Filtrar el resto: eliminar las que solapan mucho con la 'actual'
        # porque consideramos que son el mismo panel detectado varias veces
        detecciones = [
            d for d in detecciones
            if calcular_iou(actual, d) < umbral_iou
        ]

    return seleccionadas
```

Parámetros MSER

Parámetro	Valor	Descripción
delta	1	Variación máxima en MSER
min_area	5000	Área mínima de región
max_area	200000	Área máxima de región
max_variation	0.01	Variación máxima permitida
bottom_ratio	0.2	Límite inferior de relación de aspecto
top_ratio	9.0	Límite superior de relación de aspecto

resize_percentage	0.05	Expansión del bounding box (5%)
umbral_score	0.6	Umbral mínimo de score de color
umbral_iou	0.5	Umbral máximo de solapamiento (NMS)

Pipeline de detección (c_d - Color y Distribución Espacial)

El detector en `detectors/detector_color_distribucion.py` usa detección de color azul por HSV y análisis de distribución espacial con `connectedComponents`.

1. Detección de píxeles azules (HSV):
 - Rango azul: [75, 40, 40] - [180, 255, 255]
 - Conversión BGR → HSV → máscara binaria
2. Análisis de distribución espacial:
 - `cv2.connectedComponentsWithStats` para etiquetar blobs

```
def analizar_distribucion_espacial(mascara_azul):  
  
    num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(  
        mascara_azul, connectivity=8  
    )  
  
    return num_labels, labels, stats, centroids
```

3. Filtrado de candidatos:
 - Por área: `min_area_pixels=500`, `max_area_pixels=200000`
 - Por relación de aspecto: `0.15 < ratio < 10.0`

```
def obtener_candidatos_desde_blobs(num_labels, stats, centroids):

    candidatos = []

    for i in range(1, num_labels): # Empezar en 1, el 0 es el fondo
        x, y, w, h, area = stats[i]

        # Filtrar por área
        if area < min_area_pixels or area > max_area_pixels:
            continue

        # Filtrar por relación de aspecto
        if h == 0:
            continue
        ratio = w / float(h)
        if bottom_ratio < ratio < top_ratio:
            candidatos.append((x, y, w, h))

    return candidatos
```

4. Filtrado por color:

- Score de correlación con máscara ideal
- Umbral de score: 0.3

5. Agrandado de bounding boxes:

- Expansión del 10% manteniendo el centro

6. Non-Maximum Suppression (NMS):

- Eliminación de detecciones solapadas (IoU > 0.6)

7. Salida:

- resultado.txt: <nombre>;<x1>;<y1>;<x2>;<y2>;<tipo>;<score>
- resultado_imgs/cd_*.png: Imágenes con detecciones
- resultado_imgs/cd_mask_*.png: Máscara de azules (debug)

Parámetros c_d

Parámetro	Valor	Descripción
azul_bajos	[75, 40, 40]	Límite inferior HSV
azul_altos	[180, 255, 255]	Límite superior HSV
min_area_pixels	500	Área mínima de blob
max_area_pixels	200000	Área máxima de blob
bottom_ratio	0.15	Límite inferior de relación de aspecto
top_ratio	10.0	Límite superior de relación de aspecto
resize_percentage	0.10	Expansión del bounding box (10%)
umbral_score	0.3	Umbral mínimo de score de color
umbral_iou	0.6	Umbral máximo de solapamiento (NMS)

Archivos principales

- main.py - Script principal que selecciona y ejecuta el detector
- detectors/detector_mser.py - Detector MSER
- detectors/detector_color_distribucion.py - Detector por color y distribución
- evaluar_resultados.py - Evaluación de resultados (cálculo de IoU)